Technical Report: Parameter-Efficient Fine-Tuning of Small Language Models for Text-to-SQL

A Low-Rank Adaptation (LoRA) Approach on the Spider Benchmark

Author: ML Engineering & Portfolio Project | **Date:** July 2026

Abstract—*This report documents the design, implementation, and analysis of a parameter-efficient fine-tuning (PEFT) pipeline using Low-Rank Adaptation (LoRA) to adapt Qwen2.5-0.5B-Instruct for Natural Language to SQL (NL2SQL) translation. The model was trained on the cross-domain* ***Spider*** *benchmark. By training only* ***0.2184%*** *(1.08 million) of the model's parameters, we enforce structured SQL generation constrained to specific database schemas. Evaluated on 200 unseen databases from the Spider dev set, the model achieved a* ***21.5% Exact-Match (EM) accuracy****. Qualitative analysis reveals join-hallucination bias and logic reversal as primary failure modes, pointing to clear paths for future architecture scaling.*

1. Introduction & Motivation

Translating natural language questions into executable database queries (Text-to-SQL) is a cornerstone of intelligent database interaction. While large foundation models (>7B parameters) can perform zero-shot Text-to-SQL tasks, deploying them in edge or local environments is computationally expensive. This project investigates the capability of a highly compact parameter-efficient architecture—specifically, a 500-million parameter model (*Qwen2.5-0.5B-Instruct*)—fine-tuned via LoRA.

The primary research objective is to teach the model **schema-linking** (aligning natural language entities with database table/column structures) and strict output formatting (SQL-only syntax, with zero conversational filler).

2. Dataset and Preprocessing

2.1 The Spider Benchmark vs. WikiSQL

Unlike WikiSQL, which uses single-table databases and overlapping train/test schemas, **Spider** requires cross-database generalization. In Spider, the databases in the validation set have no overlap with those in the training set. To generalize, the model must read the schema dynamically, identify primary-foreign key relationships, and link them to the question.

2.2 Data Pipeline Implementation

Since Spider's base release does not contain raw text representations of schemas, we integrate a helper schema dataset *richardr1126/spider-schema*. The preprocessing pipeline is implemented in the Jupyter notebook (*src/fine-tune-kaggle.ipynb*) through the following functions:

1. **format_schema(db_id, schema_lookup)**: Extracts the list of tables, columns, and foreign key rules for a database.
2. **build_prompt(question, schema_text)**: Combines the schema context, the user question, and formatting rules. It

appends the instruction: *'Respond with only the SQL query. No explanation, no markdown formatting.'*
3. **format_example(row, schema_lookup)**: Structures the inputs into a standard multi-turn ChatML conversation schema.

3. Low-Rank Adaptation (LoRA) Methodology

3.1 Mathematical Principles

Fine-tuning all 495 million weights of Qwen2.5-0.5B requires large GPU memory. LoRA solves this by freezing the pre-trained weights and representing the weight update as a low-rank factorization of two trainable matrices:

$$\Delta W = B \cdot A$$

where B and A are low-rank matrices. The forward pass becomes: $h = W_0 x + (\alpha/r)(B \cdot A) x$.

3.2 Hyperparameter Configuration

We applied the following Peft configuration to target the key attention matrices:
• **Rank (r):** 16 (controls the bottleneck dimension)
• **LoRA Alpha (α):** 32 (scaling coefficient)
• **Dropout:** 0.05 (preventing co-adaptation)
• **Target Modules:** *['q_proj', 'v_proj']* (restricting adaptation to Query and Value projections)
• **Trainable Parameters:** 1,081,344 / 495,114,112 (**0.2184%**)

```
# LoRA Configuration used in PEFT
lora_config = LoraConfig(
r=16,
lora_alpha=32,
lora_dropout=0.05,
target_modules=['q_proj', 'v_proj'],
bias='none',
task_type="CAUSAL_LM"
)
```

4. Training Pipeline & Overfitting Analysis

4.1 Configuration

Training was performed using the Hugging Face *trl* library's *SFTTrainer*:
• **Dataset Size:** 7,000 training examples.
• **Effective Batch Size:** 8 (per-device batch size of 4, gradient accumulation steps of 2).
• **Learning Rate:** 2e-4.
• **Loss Masking:** SFT loss was evaluated strictly on assistant turn tokens, ignoring prompt schemas.

4.2 Overfitting Identification

Eval loss rose starting at step 100, while train loss kept falling (confirming overfitting). Training was stopped early and the step-100 weights were restored.

Step	Training Loss	Validation Loss
100	0.6205	0.8678 (Best checkpoint)
200	0.4622	0.9109
300	0.4064	1.0753

500	0.2247	1.2173
800	0.1771	1.4191

5. Detailed Code Walkthrough

5.1 The Jupyter Training Notebook (fine-tune-kaggle.ipynb)
Contains the complete training pipeline. It handles downloading/formatting schemas, applying ChatML prompt wrappers, setting up the LoRA configuration, and running supervised training using the SFTTrainer.

5.2 CLI Inference Utility (inference.py)
Enables testing queries without loading full training modules. It loads the base causal LM and applies the LoRA adapter using PEFT's wrapper. It supports running predictions via direct CLI parameters.

```
# Load base and attach LoRA adapter dynamically
base_model = AutoModelForCausalLM.from_pretrained(
"Qwen/Qwen2.5-0.5B-Instruct",
torch_dtype=torch.float16,
device_map="auto"
)
model = PeftModel.from_pretrained(
base_model,
"thefounder03/nl2sql-lora-qwen2.5-0.5b"
)
```

6. Evaluation Metrics & Results

On 200 held-out databases from the Spider dev set, the model achieved a **21.5% Exact-Match (EM) accuracy**. Exact-Match is extremely strict, checking character-by-character equivalence. It penalizes semantically valid SQL statements if they vary in capitalization, join ordering, or aliases from the ground truth.

7. Qualitative Error Analysis & Case Studies

7.1 Join-Hallucination Bias

Because complex join queries dominate the training distribution, the model often adds unnecessary `JOIN` statements on prompts that only require a single table.

```
-- Question: Find the name of all singers.
-- Reference SQL:
SELECT name FROM singer;

-- Model Output (Hallucinated JOIN):
SELECT T1.name FROM singer AS T1
JOIN concert AS T2 ON T1.singer_id = T2.singer_id;
```

7.2 Reversed Logic/Constraint Errors

The small base capacity (0.5B parameters) limits logical semantics, causing the model to swap sorting operators (e.g., ordering oldest-to-youngest using ASC instead of DESC).

```
-- Question: List names and ages of singers from oldest to youngest.
-- Reference SQL:
SELECT name, age FROM singer ORDER BY age DESC;

-- Model Output (Reversed Logic):
SELECT name, age FROM singer ORDER BY age ASC;
```

8. Discussion & Key Learnings

LoRA fine-tuning effectively handles structured output constraints (forcing the model to output SQL syntax only, bypassing chatty instructions) and improves schema entity linking. However, it does not solve core reasoning failures or eliminate structural hallucinations. To overcome these limitations, future work will scale parameters to Qwen2.5-7B using QLoRA and switch evaluation to execution-based metrics.